

Task 23:

Apply Vibe Coding and Prompt Engineering techniques using a Generative AI tool. Design effective prompts to generate code snippets, cloud configuration templates, or application logic, and evaluate the quality and efficiency of the generated outputs for real-world business applications.

Aim

To apply **Vibe Coding and Prompt Engineering techniques** using a Generative AI tool to design effective prompts that generate **code snippets, cloud configurations, and application logic**, and evaluate the quality and efficiency of outputs.

Algorithm

1. Select a Generative AI tool (e.g., ChatGPT).
2. Define the problem clearly (e.g., REST API, cloud config).
3. Write an **initial prompt**.
4. Generate output (code/config).
5. Evaluate output correctness and quality.
6. Refine the prompt (add constraints, clarity).
7. Generate improved output.
8. Compare outputs (before vs after refinement).
9. Document prompt versions and results.
10. Analyze efficiency and usability.

Implementation

1. Initial Prompt (Basic)

Generate a Spring Boot REST API for managing products.

Output

Refined

Generate a Spring Boot REST API with CRUD operations for Product entity using JPA, MySQL, and proper layered architecture (Controller, Service, Repository).

Output

```

@Entity
public class Product {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    private double price;
}

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping
    public List<Product> getAll() {
        return service.getAllProducts();
    }
}

```

}Generate a production-ready Spring Boot REST API for Product Management with:

- CRUD operations
- Validation
- Exception handling
- MySQL configuration
- Layered architecture
- Sample JSON responses

Output:

```

1  ---
2  model: gpt-3.5-turbo
3  temperature: 0.5
4  ---
5
6  # Generation unit test
7
8  You are a Java programmer, and you are developing a Spring Boot application.
9  The tech stack is Java 17, Spring Boot 3.0, JUnit 5 , Spring Boot Test, AssertJ etc. (.system)
10
11 Please write unit test with Spring Boot Test for the below code :
12
13 ```java
14
15 package org.mvnsearch.service;
16
17 import org.springframework.stereotype.Component;
18
19 @Component
20 public class UserService {
21     public User findNickById(long id) {
22         return "Jackie";
23     }
24 }
25 ```
26
27 #### ChatGPT
28
29 Sure, here's an example of a unit test using Spring Boot Test for the 'UserService' class:
30
31 ```java
32 package org.mvnsearch.service;
33
34 import org.junit.jupiter.api.Test;
35 import org.springframework.beans.factory.annotation.Autowired;
36 import org.springframework.boot.test.context.SpringBootTest;
37
38 @SpringBootTest
39 class UserServiceTest {
40     @Autowired
41     private UserService userService;
42
43     @Test
44     void findNickById() {
45         User user = userService.findNickById(1L);
46         assertEquals("Jackie", user);
47     }
48 }
49 ```

```

Result

Successfully applied prompt engineering techniques Improved code generation quality through refinement Generated scalable and production-ready outputs Understood importance of clear and detailed prompts